

Reviewer Pack — Kronecker Coefficient Non-Zero Threshold for Permanent-Compl...

Entry c93b44a7d21e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 15:10:36 UTC

Kronecker Coefficient Non-Zero Threshold for Permanent-Complete SAT Instances

Entry ID: c93b44a7d21e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 15:10:36 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Symmetric Groups **Field B** (complexity object): SAT Instances

Statement:

For a random 3-CNF formula Φ with n variables and m clauses, let $\lambda = (\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_k)$ be the partition encoding the clause-variable incidence matrix's row sums. Define $k(\Phi) = \sum_{i=1}^k \lambda_i$. Then, the Kronecker coefficient $g(\lambda, \mu, \nu)$ for partitions $\mu = (n)$, $\nu = (m)$ is non-zero iff Φ is unsatisfiable, and $k(\Phi) \leq 2n \log n$.

Rationale (proposer's reasoning):

Kronecker coefficients govern tensor product decompositions of symmetric group representations. By encoding SAT instances as incidence matrices, their combinatorial structure maps to partition tuples. Non-zero Kronecker coefficients imply representation-theoretic entanglement that may correlate with computational hardness, offering a bridge between algebraic combinatorics and SAT complexity.

Taxonomy category: GCT_DET_PERM (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 55cf2f25b47d7e4f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations, permutations

def generate_3cnf(n: int, m: int) -> list:
    variables = set(range(1, n + 1))
    clauses = []
    for _ in range(m):
        clause = random.sample(variables, 3)
        sign = random.choice([-1, 1])
        clauses.append((sign, tuple(sorted(clause))))
    return clauses

def is_satisfiable(clauses: list) -> bool:
    def dfs(model, i):
        if i == len(clauses):
            return True
        for assignment in [True, False]:
            model[clauses[i][1]] = assignment
            if all((model[var] ^ sign) for sign, var in clauses[i]):
                if dfs(model, i + 1):
                    return True
        return False

    n = max(max(clause[1]) for clause in clauses)
    model = [None] * (n + 1)
    return dfs(model, 0)

def partition_to_tuple(partition: list) -> tuple:
    return tuple(sorted(partition, reverse=True))

def kronecker_coefficient(lambda_, mu, nu):
    if len(mu) != len(nu):
        return 0
    n = len(mu)
    p = [min(lambda_[i], mu[i], nu[i]) for i in range(n)]
    q = [lambda_[i] - p[i] for i in range(n)]
```

```

r = [mu[i] - p[i] for i in range(n)]
s = [nu[i] - p[i] for i in range(n)]

def sign(p, q, r, s):
    return sum((p[i] - q[i]) * (r[i] - s[i]) for i in range(n)) % 2

def partition_to_int(partition):
    return sum(partition[i] * math.factorial(i) for i in range(len(partition)))

def int_to_partition(num, n):
    partition = [0] * n
    factorial = 1
    for i in range(n - 1, -1, -1):
        if num >= factorial:
            partition[i] = (num // factorial) + 1
            num %= factorial
            factorial //= (i + 1)
    return partition

def kronecker(p, q, r, s):
    if p == q and q == r and r == s:
        return 1
    if len(set(p)) != len(p) or len(set(q)) != len(q) or len(set(r)) != len(r) or len(set(s)) != len(s):
        return 0
    if sum(p) != sum(q) or sum(q) != sum(r) or sum(r) != sum(s):
        return 0
    if any(x < 0 for x in p + q + r + s):
        return 0

    p_int = partition_to_int(p)
    q_int = partition_to_int(q)
    r_int = partition_to_int(r)
    s_int = partition_to_int(s)

    if p_int > q_int or q_int > r_int or r_int > s_int:
        return 0

    if sign(p, q, r, s) == 1:
        return (-1) ** (sum(x * math.factorial(i) for i, x in enumerate(q)) - sum(x * math.factorial(i) for i, x in enumerate(s)))
    else:
        return 0

    return kronecker(p, q, r, s)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    m = random.randint(2 * n, 4 * n)
    clauses = generate_3cnf(n, m)
    lambda_ = [sum(clause.count(var) for clause in clauses) for var in range(1, n + 1)]
    k_lambda = sum(lambda_)
    mu = (n,)
    nu = (m,)
    g_lambda_mu_nu = kronecker_coefficient(lambda_, mu, nu)

    is_unsat = not is_satisfiable(clauses)

```

```

conjecture_holds = is_unsat and k_lambda <= 2 * n * math.log(n) or not is_unsat and g_lambda_n
counterexample = "mapping_undefined" if not conjecture_holds else ""

return {
    "metric_name": "Kronecker Coefficient",
    "metric_value": g_lambda_mu_nu,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2c8df4b8.py", line 122, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2c8df4b8.py", line 98, in run_trial
    clauses = generate_3cnf(n, m)
              ^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2c8df4b8.py", line 21, in generate_3cnf
    clause = random.sample(variables, 3)
              ^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/usr/lib/python3.12/random.py", line 413, in sample
    raise TypeError("Population must be a sequence. ")

```

TypeError: Population must be a sequence. For dicts or sets, use sorted(d).

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to invalid population type in random.sample; unable to evaluate conjecture's validity | next: Fix generate_3cnf() to use ordered sequences (e.g., lists) for variables and re-run tests

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	58909
2	propose	ollama_remote	qwen3:8b	0	0	43642
3	preregistration	ollama_remote	qwen3:8b	0	0	24255
4	novelty	ollama_remote	qwen3:8b	0	0	20681
5	novelty	ollama_remote	qwen3:8b	0	0	18852
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21387
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15283
8	judge	ollama_remote	qwen3:8b	0	0	17214

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 220222 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/c93b44a7d21e.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/c93b44a7d21e.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/c93b44a7d21e.tar.gz (if generated)